

LBIST Project — File-by-File Explanation

Deepak Chaudhary

September 14, 2026

This document describes every file in the LBIST project directory, grouped by purpose. Each entry lists what the file is and its role in the design flow.

1 RTL Source Files

`pattern_gen.sv`

Test Pattern Generator

The Test Pattern Generator (TPG). A 4-bit maximal-length Linear Feedback Shift Register (LFSR) with taps `out[3] ⊕ out[0]` and seed 0001. Automatically produces the 15 test vectors that exercise the Circuit Under Test, achieving full period $2^4 - 1 = 15$.

`cut.sv`

Circuit Under Test

The combinational circuit being tested. Implements `out = (in[0] & in[1]) | (in[2] & in[3])`, which synthesizes to 3 NAND gates. This is the DUT for the entire BIST flow.

`stuck_models.sv`

Fault-Injection Model

A combinational model that simultaneously produces the golden output and 12 stuck-at-fault variants (A, B, C, D, X, Y each at 0 and 1). The fault to be exercised is selected by the 4-bit `smodel` input. Used both for RTL fault simulation and as a reference for the industrial fault simulator.

`MISR.sv`

Multi-Input Signature Register (ORA)

The Output Response Analyzer. Contains 13 parallel 4-bit MISRs — one for the golden output and one for each of the 12 faulty responses. Each MISR compresses a 15-bit response stream into a 4-bit signature using taps at bits 0 and 2. Total: 52 flip-flops and 26 XOR gates.

`comparator.sv`

Signature Comparator

Combines the fault signatures with the golden signature. Emits 12 detection flags (`Det_A` through `Det_Ybar`) and a 4-bit `num_detected` count. Built entirely from XNOR-based equality checks.

`controller.sv`

BIST Controller FSM

A 4-state Finite State Machine (`IDLE` → `RUN` → `EVAL` → `DONE`) that sequences the entire test. Emits `start` (enables the LFSR), `scan_en`, `capture_en`, `done`, `pass`, and a 7-bit `coverage` output. Includes a 5-bit cycle counter that advances once per LFSR pattern.

`scan_ff.sv`

Scan Flip-Flop

A DFT-ready flip-flop with three modes: *shift*, *capture*, and *hold*. Provides serial chain access via `TDI/TDO`. Foundation for scan-chain integration.

`scan_reg.sv`

Scan Chain

A chain of `REG_SIZE` scan flip-flops connected in series. Allows parallel capture and serial shift-out of internal state. Part of the DFT infrastructure but not yet wired into the top-level BIST.

`d_ff.sv`

D Flip-Flop Primitive

A basic D flip-flop with asynchronous active-low reset. Used as a building block by the scan flip-flop.

`top.sv`

BIST Top-Level

The top-level BIST integration. Wires together the LFSR, CUT, fault model, MISR array, comparator, and controller into a single synthesizable hierarchy. Exposes the test result (`pass`, `done`, `coverage`) and the CUT signals for observation.

`tb.sv`

Testbench

A self-checking testbench that drives clock, reset, and `start_ext`, loops through all 12 hand-modeled faults, waits for each run to complete, and reports per-fault detection results plus overall coverage. Also dumps a VCD waveform for inspection.

2 Synthesis and Netlist Files

`cut_netlist.v`

Synthesized CUT Netlist

The gate-level netlist of `cut.sv` produced by Yosys. Contains 3 `$_NAND_` primitive instances. This is the input to the industrial fault simulator.

netlist.v *Full BIST Netlist*

The complete gate-level netlist of the entire BIST top, also produced by Yosys. Confirms that the whole design is synthesizable without errors (~380 cells).

simcells.v *Yosys Cell Simulation Library*

A Verilog library shipped with Yosys that defines behavioral models for internal cells (`$_AND_`, `$_NAND_`, `$_DFF_PNO_`, etc.). Passed to AUCOHL/Fault as the cell model so it understands each gate.

simlib.v *Yosys Full Simulation Library*

A larger library covering complex cells (`$add`, `$mux`, `$dff`, `$mem`, etc.). Used for detailed RTL netlist simulation when the design contains higher-level primitives.

3 Fault Simulation Artifacts (AUCOHL/Fault)

cut_netlist.v.tv.json *Auto-Generated Test Vectors*

The industrial simulator's internally generated test patterns, encoded in JSON. Includes each vector, the golden output, and per-vector coverage metadata. Used to establish the baseline coverage.

cut_netlist.v.tv.svf *Serial Vector Format Output*

The same test vectors exported in SVF (Serial Vector Format) — a standard ATE-compatible format used to drive physical tester hardware.

faults.yml *Exhaustive Fault List*

The complete list of fault sites enumerated by Fault in the synthesized CUT: 14 sites across the 3 NAND gates, inputs, and output of the circuit.

covered.yml *Baseline Coverage Report*

The coverage report produced when Fault runs with its own internal random pattern generator. Achieves 100% coverage across all 14 fault sites (28 stuck-at faults).

lfsr_patterns.json *LFSR Patterns Encoded for Fault*

The 15 patterns produced by our LFSR, re-encoded into Fault's JSON test-vector format. Fed to the simulator via the `-externalTVSet` option.

lfsr_covered.yml *LBIST Coverage Report*

The coverage report produced when Fault replays our LFSR's 15 patterns. Achieves 100% coverage — matching the baseline, which proves the LFSR sequence is sufficient.

parsetab.py *Auto-Generated Parser Table*

A Python parser table auto-generated by AUCOHL/Fault's internal Yacc-based parser. Produced as a side effect of running the tool; not part of the design.

4 Simulation and Waveform Files

bist.vcd *Waveform Dump*

A Value Change Dump (VCD) capturing every signal transition during testbench simulation. Viewable in GTKWave or Surfer. Contains the full IDLE→RUN→EVAL→DONE FSM trace, LFSR patterns, MISR signatures, and detection flags.

build.log *Compilation Log*

Output from the Icarus Verilog compile step. Useful for debugging synthesis or elaboration issues.

tb.vvp *Compiled Testbench Binary*

The compiled simulation binary for the testbench, produced by `iverilog` and executed by `vvp`. Contains the elaborated design plus the testbench.

top.vvp *Compiled BIST Binary*

The compiled simulation binary for the top-level BIST design alone (without the testbench wrapper).

5 Summary

Category	Count	Purpose
RTL source files	10	Original SystemVerilog design
Synthesis / netlists	4	Gate-level outputs + cell libraries
Fault-sim artifacts	7	Coverage reports, patterns, fault lists
Simulation / waveform	4	Compiled binaries, VCD, build log

Total: 25 files covering the complete LBIST design, synthesis, verification, and fault-coverage evaluation flow.